K.Harsha Vardhan

192425228

MLA0201-FUNDAMENTAL OF MACHINE LEARNING

Annexure A

Pseudocode Writing Task (Algorithm Design)

Question 1: Monte Carlo Sampling for Probability Estimation

A logistics company wants to estimate the probability of on-time deliveries under uncertain traffic conditions using **Monte Carlo Sampling**.

Task:
Write a pseudocode algorithm to:

- Generate random samples representing delivery conditions.
- Simulate multiple delivery trials.
- Estimate the probability of on-time delivery.
- Display the estimated probability after all simulations.

Question 2: ε-Greedy Algorithm for K-Armed Bandit

An online streaming platform recommends one of several movies to maximize user engagement. The platform must balance **exploration** and **exploitation**.

Task:
Write a pseudocode algorithm to:

- Initialize rewards for all movie recommendations.
- Implement the **ε-greedy exploration strategy**.
- Update reward estimates after each user interaction.
- Identify the movie with the highest expected reward.

Question 3: Value Iteration for Robot Path Planning

A warehouse robot must determine the optimal path to reach a destination while maximizing rewards and avoiding obstacles.

Task:
Write a pseudocode algorithm to:

- Initialize the value function for all states.

- Update state values using the **Bellman optimality equation**.
- Repeat the process until convergence.
- Output the optimal value function and policy.

Question 4: Temporal Difference (TD) Learning for Game Score Prediction

A game AI agent learns the value of each game state based on rewards received after every move.

Task:
Write a pseudocode algorithm to:

- Initialize state-value estimates.
- Observe the current state, reward, and next state.
- Update the state value using the **Temporal Difference (TD) learning rule**.
- Continue until all game episodes are completed.

Question 5: Policy Iteration for Autonomous Drone Navigation

An autonomous drone must learn the optimal navigation policy to deliver packages while minimizing energy consumption.

Task:
Write a pseudocode algorithm to:

- Initialize a random policy.
- Perform **policy evaluation** to compute state values.
- Improve the policy by selecting the best action for each state.
- Repeat until the policy becomes stable and output the optimal policy.

Question 1: Monte Carlo Sampling for Probability Estimation
Algorithm: Monte Carlo Sampling
START

Set total_trials = N
Set on_time_count = 0

```
FOR i = 1 TO N
   Generate random traffic condition
   Simulate delivery

   IF delivery is on-time THEN
      on_time_count = on_time_count + 1
   END IF
END FOR

probability = on_time_count / total_trials

Display probability

END
```

Formula:
Estimated Probability = Number of On-Time Deliveries / Total Deliveries

---

Question 2: ε-Greedy Algorithm for K-Armed Bandit Algorithm: ε-Greedy Movie Recommendation START

```
Set K = number of movies
Set epsilon = ε
Set reward[movie] = 0 for all movies
Set count[movie] = 0 for all movies
FOR each user interaction

   Generate random number r

   IF r < epsilon THEN

      Select a random movie       // Exploration
   ELSE
      Select movie with highest reward estimate // Exploitation
   END IF

   Observe user reward

   count[movie] = count[movie] + 1
   reward[movie] = reward[movie] +
               (user_reward - reward[movie]) / count[movie]

END FOR

Select movie with highest reward estimate
```

```
Display best movie

END
```

Main idea:

- **Exploration:** Try different movies.
- **Exploitation:** Recommend the movie with the highest estimated reward.

Question 3: Value Iteration for Robot Path Planning
Algorithm: Value Iteration

```
START

Initialize value V(s) = 0 for all states
Set discount factor γ
Set threshold θ

REPEAT

   Set Δ = 0

   FOR each state s

      IF s is an obstacle THEN
         Skip state
      END IF

      old_value = V(s)

      V(s) = maximum over all actions [
            Reward(s,a) + γ * V(next_state)
            ]

      Δ = maximum(Δ, |old_value - V(s)|)

   END FOR

UNTIL Δ < θ

FOR each state s
   Select action with highest value
   Store action as policy[s]
END FOR

Display V(s)
Display policy

END
```

Main idea: Update each state using the **Bellman optimality equation** until the values stop changing.

Question 4: Temporal Difference (TD) Learning for Game Score Prediction
Algorithm: TD Learning

START

Initialize V(s) = 0 for all states
Set learning rate α
Set discount factor γ

FOR each game episode

  Observe initial state S

  WHILE game is not over

    Perform an action
    Observe reward R
    Observe next state S'

    V(S) = V(S) + α * [R + γ * V(S') - V(S)]

    S = S'

  END WHILE

END FOR

Display state values V(s)

END

TD Learning Formula:

$V(S) = V(S) + \alpha [R + \gamma V(S') - V(S)]$

Question 5: Policy Iteration for Autonomous Drone Navigation Algorithm: Policy Iteration

START

Initialize a random policy π(s)
Initialize V(s) = 0 for all states

REPEAT

  // Policy Evaluation
  REPEAT

```
        FOR each state s
            V(s) = Reward(s, π(s)) +
                  γ * V(next_state)
        END FOR
    UNTIL values converge

    // Policy Improvement
    policy_stable = TRUE

    FOR each state s

        old_action = π(s)

        Select action with highest:
            Reward(s,a) + γ * V(next_state)

        π(s) = best action

        IF old_action != π(s) THEN
            policy_stable = FALSE
        END IF

    END FOR

UNTIL policy_stable = TRUE

Display optimal policy

END
```